

优化问题——中八摆球方案

众所周知，中式八球总计15颗球，其中7颗纯色球，7颗双色球，1颗纯黑色8号球，该球不算在 7/7 之中。

众所周知，开球时所有球摆成三角形，其中黑8固定在第三排中间的位置，其余球的颜色尽可能分隔开摆放。我们的目标就是找出最佳摆放方法。

显然，固定黑8后还剩14个位置放球，那么总共自然就有 $C_{14}^7 = 3432$ 种可能性，这对计算机而言是相当小的数据了，那么我们选择交给计算机来解决这个问题。

第一步是想办法构造出能够描述台球摆放位置与颜色的数据类型，这可以通过类轻松实现，我们先来写一个台球类：

```
class pool_pos:
    # 此类描述台球开球前的初始三角位置

    def __init__(self, pos_row: int, pos_col: int, value: int):
        self.pos_row = pos_row
        self.pos_col = pos_col
        self.value = value

        self.adjoint = []

    def add_adjoint_pos(self, pos: 'pool_pos'):
        """
        这个函数在此次项目中并没有起到实质性的作用，只是为了完善结构而写。
        :param pos:
        :return:
        """
        self.adjoint.append(pos)
        pos.adjoint.append(self)

    def add_search_pos(self, pos: 'pool_pos'):
        """
        相较于上一个函数，这个方法单向添加元素，用于主文件中的搜索方法，而非相邻结构的完善。
        :param pos: 单向添加的相邻球
        :return: None
        """
        self.search.append(pos)
```

该类包含了台球在三角中的行、列、颜色与相邻球信息，这些信息应该足以用来解决问题了。

有了类之后，下一步自然就是类的实例化——真正地弄些台球出来。为了实现这一点，我们来制作一个函数工具：

```
def sort_balls() → list:
    """
    该函数用来设定台球的初始位置，相邻关系，以及初始值（-1代表默认均为黑球）
    :return: list
    """
    balls = []
    for i in range(5): # 构造台球的三角摆放形状
        for j in range(i + 1):
            balls.append(pool_pos(i + 1, j + 1, -1))

    # 构建台球之间的相邻关系
    for ball in balls:
        lvl = ball.pos_row
        col = ball.pos_col
        right = col + 1

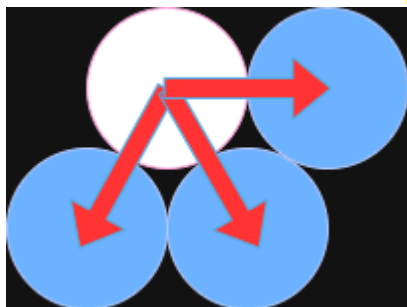
        if right ≤ lvl:
            founded_index = next((index for index, b in
            enumerate(balls) if b.pos_row == lvl and b.pos_col == right), None)
            ball.add_search_pos(balls[founded_index])

        # 寻找下一层的两颗相邻球
        if lvl < 5:
            founded_index_1 = next((index for index, b in
            enumerate(balls) if b.pos_row == lvl + 1 and b.pos_col == col), None)
            founded_index_2 = next((index for index, b in
            enumerate(balls) if b.pos_row == lvl + 1 and b.pos_col == col + 1),
            None)

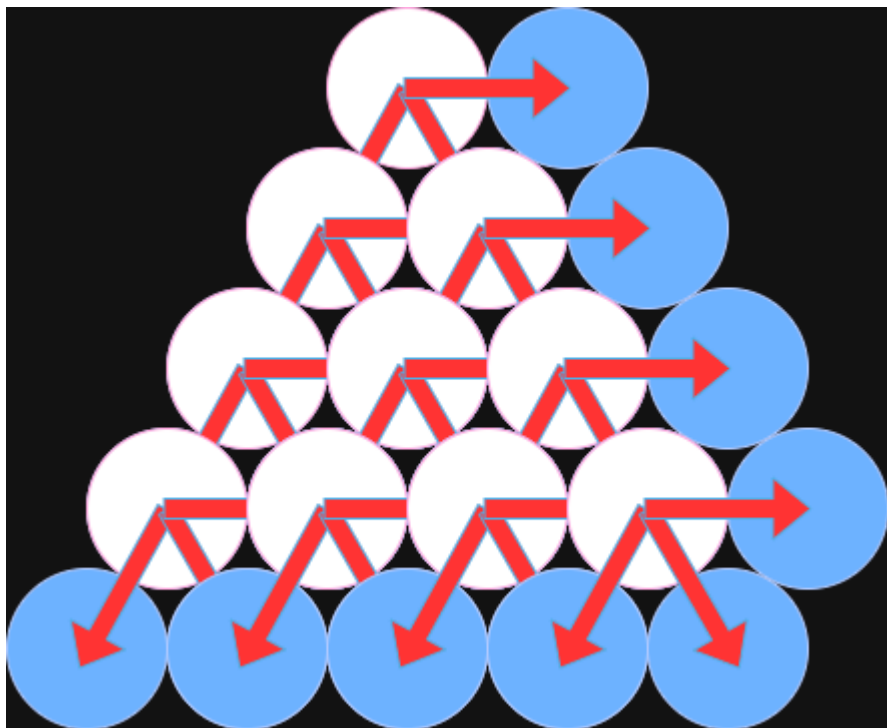
            ball.add_search_pos(balls[founded_index_1])
            ball.add_search_pos(balls[founded_index_2])
        else:
            continue

    return balls
```

此函数的核心思路在于如何确定一颗球的相邻球，此处我用的搜索模式如下所示：



即考察一颗球右边一个位置与下方两个位置的球，其中白球是我们当前考察的球，蓝色为搜索位置。容易发现该模式可迭代，迭代10次后就得到了全部的相邻关系：



通过这个函数，我们就可以把15颗球造出来并摆好了：

```
pool_tri = sort_balls()
```

其中 `pool_tri` 是存放这15颗球的列表。此时如果我们查询 `pool_tri[0]` 将得到三角形最上方那颗球的信息，即

```
pool_tri[0].pos_row = 1
pool_tri[0].pos_col = 1
pool_tri[0].value = -1
pool_tri[0].adjoint = [pool_tri[1], pool_tri[2]]
```

在台球类中，我使用 `value` 属性来描述颜色信息，`-1` 表示黑色，`0` 表示纯色，`1` 则为双色。让计算机枚举全部3432个方案并选出最优，自然需要给它提供优化目标，这个问题的优化目标很简单：如果同色球相邻就是不好的，我们要从数学描述上给出惩罚，否则平安无事。在任意给定的一个方案中，对其中任一颗球 x 而言，其对整体方案的贡献可以用很简单的函数来描述：

$$P_x(y) = \begin{cases} 0, & \text{if } x \neq y; \\ -1, & \text{if } x = y. \end{cases}$$

其中之所以选取 0 而不是正数是为了避免正数与 -1 相互抵消。这样一来整体的损失函数就可以定义成

$$\mathcal{L} = \sum_{x,y} P_x(y) = \sum_x \sum_y P_x(y).$$

我们的目标即为

$$\max \mathcal{L}.$$

于是接下来就需要写一个分数计算器：

```
def score_counter(stage: list) → int:
    score = 0
    for ball in stage:
        lvl = ball.pos_row
        col = ball.pos_col
        right = col + 1

        if right ≤ lvl:
            founded_index = next((index for index, b in
enumerate(stage) if b.pos_row == lvl and b.pos_col == right), None)
            if founded_index is not None and ball.value ==
stage[founded_index].value:
                score -= 1

        if lvl < 5:
            index_1 = next((index for index, b in enumerate(stage) if
b.pos_row == lvl + 1 and b.pos_col == col), None)
            index_2 = next((index for index, b in enumerate(stage) if
b.pos_row == lvl + 1 and b.pos_col == col + 1), None)
            if index_1 is not None and ball.value ==
stage[index_1].value:
                score -= 1
            if index_2 is not None and ball.value ==
stage[index_2].value:
                score -= 1

    return score
```

其中用到了同上一个函数相同的搜索模块，只是这次是考察相邻位置球的颜色是否与当前的球相同，如果相同就扣一分，否则略过。

.....

但是真的需要这么麻烦吗？ 注意我们在台球类中构建的 `search` 列表，对该列表的搜索完全符合我们刚才提出的搜索模式，既然如此用这个列表来找相邻球就可以了。我们实际上使用以下计算器：

```
def new_score_counter(stage: list) → int:
    score = 0
    for ball in stage:
        for target in ball.search:
            if target.value == ball.value:
                score -= 1
    return score
```

由于我们初始将所有球的颜色值都取为-1(黑色)，分数计算器自然需要我们输入一个真实的颜色状态，因此我们缺少的最后一个工具就是对所有颜色方案(摆放方案)的遍历：

```
def generate_complex_modified_lists(
    original_list: List[Any],
    k: int,
    fixed_index: int,
    main_change_value: Any = 'X',
    other_change_value: Any = '0'
) → Generator[List[Any], None, None]:
```

"""

遍历列表所有可能性，其中：

- 一个位置的元素被固定。
- `k` 个其他位置的元素被改变成一个值。
- 其余非固定位置的元素被改变成另一个值。

参数：

`original_list` (`List[Any]`)：原始列表。

`k` (`int`)：需要改变为 `main_change_value` 的元素个数。

`fixed_index` (`int`)：被固定的元素索引，该位置的值不会改变。

`main_change_value` (`Any`)：`k` 个选中位置要变成的新值。

`other_change_value` (`Any`)：其余位置要变成的新值。

返回：

`Generator[List[Any], None, None]`：一个生成器，每次迭代返回一个修改后的新列表。

"""
n = len(original_list)

```

# --- 输入验证 ---    if not 0 ≤ fixed_index < n:
    print(f"错误: fixed_index ({fixed_index}) 超出列表范围 [0, {n - 1}].")
    return
# 可选位置的数量是 n-1    if not 0 ≤ k ≤ n - 1:
    print(f"错误: k ({k}) 的值必须在 0 到 {n - 1} (n-1) 之间, 因为有一个位置被固定了.")
    return

# 1. 创建一个不包含 fixed_index 的“可选索引”列表
selectable_indices = [i for i in range(n) if i ≠ fixed_index]

# 2. 从“可选索引”中找出所有 k 个索引的组合
for index_combination in itertools.combinations(selectable_indices, k):
    # 为了方便查找, 将元组转换为集合
    selected_indices_set = set(index_combination)

    # 3. 创建一个新列表来存放结果, 避免修改原始列表
    modified_list = original_list.copy()

    # 4. 遍历所有原始索引, 根据其分类填充新列表
    for i in range(n):
        if i == fixed_index:
            # 类别一: 固定位置
            modified_list[i] = original_list[i]
        elif i in selected_indices_set:
            # 类别二: 被选中的 k 个位置
            modified_list[i].value = main_change_value
        else:
            # 类别三: 其他位置
            modified_list[i].value = other_change_value

    yield modified_list

```

这个函数可以固定黑8的颜色不变, 让我们获取 7/7 配色的全部可能摆法, 最后返回一个包含全部方案的列表, 这个列表具体是:

```

modified_generator = generate_complex_modified_lists(
    pool_tri,
    k=7,
    fixed_index=4,
    main_change_value=0,

```

```
        other_change_value=1
    )
```

最后，我们只需要知道这些方案中得分最高的是哪一个，以及最高分具体是多少即可：

```
final_result = []
final_score = -999
turn = 0
for result in modified_generator:
    if turn == 0:
        final_result = copy.deepcopy(result)
        final_score = new_score_counter(result)
        turn = 1
    else:
        new_score = new_score_counter(result)
        if new_score >= final_score:
            final_result = copy.deepcopy(result)
            final_score = new_score

show_list = []
for item in final_result:
    show_list.append(item.value)

print(show_list)
print(final_score)
```

⚠ Warning

对 `result` 一定要使用 `deepcopy()` !!! 切勿导致指向错误。

最终得到的最佳方案得分为 -7 ，因此可知在本文的损失函数构造下，最优方案一定是 -7 分，但是最优方案并不唯一。

最后我们提供两个最优方案作为本篇的结束：

